Long Short-Term Memory Networks

A Mathematical Treatment

A Rigorous Formulation of LSTM Architecture,

Gating Mechanisms, and Learning Algorithms

January 25, 2026

Abstract

This document provides a rigorous mathematical treatment of Long Short-Term Memory (LSTM) networks. We present the complete mathematical formulation of LSTM architectures, including detailed derivations of the gating mechanisms, cell state dynamics, and the backpropagation through time algorithm. The treatment emphasizes the mathematical foundations that enable LSTMs to capture long-range dependencies in sequential data, with particular attention to the gradient flow properties that distinguish LSTMs from vanilla recurrent neural networks.

Contents

1 Introduction and Motivation **2**

2 Mathematical Preliminaries **2**
2.1 Notation . . . 2
2.2 Activation Functions . . . 2
2.3 Hadamard Product . . . 3

3 LSTM Cell Architecture **3**
3.1 The Forget Gate . . . 3
3.2 The Input Gate . . . 4
3.3 Cell State Update . . . 4
3.4 The Output Gate . . . 5

4 Complete Forward Pass **5**
4.1 Summary of LSTM Equations . . . 5
4.2 Parameter Count . . . 5
4.3 Initial Conditions . . . 6

5 Compact Matrix Formulation **6**

6 Backpropagation Through Time **6**
6.1 Gradient with Respect to Hidden State . . . 7
6.2 Gradient with Respect to Cell State . . . 7
6.3 Gate Gradients . . . 7
6.4 Weight Gradients . . . 8

7 LSTM Variants **8**
7.1 Peephole Connections . . . 8
7.2 Gated Recurrent Unit (GRU) . . . 8
7.3 Coupled Input-Forget Gate . . . 9

8 Bidirectional LSTM **9**

9 Multi-Layer (Stacked) LSTM **9**

10 Conclusion **9**

1 Introduction and Motivation

The Long Short-Term Memory (LSTM) architecture, introduced by Hochreiter and Schmidhuber [1], addresses the fundamental problem of learning long-range dependencies in recurrent neural networks. Standard RNNs suffer from the *vanishing gradient problem*, where gradients decay exponentially during backpropagation through time, making it difficult to learn dependencies spanning many time steps.

Definition 1.1 (Sequential Data)**.** Let $\mathcal{X} \subseteq \mathbb{R}^n$ be an input space. A sequence of length T is an ordered tuple $\mathbf{x}_{1:T} = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T)$ where each $\mathbf{x}_t \in \mathcal{X}$ for $t \in \{1, 2, \ldots, T\}$.

The LSTM maintains two state vectors that evolve through time:

- The **cell state** $\mathbf{c}_t \in \mathbb{R}^m$, which serves as long-term memory
- The **hidden state** $\mathbf{h}_t \in \mathbb{R}^m$, which serves as short-term memory and output

The key innovation of the LSTM is its gating mechanism, which controls information flow through the network and allows gradients to propagate over long sequences without vanishing.

2 Mathematical Preliminaries

2.1 Notation

Throughout this document, we adopt the following notation:

Symbol	Description
$\mathbf{x}_t \in \mathbb{R}^n$	Input vector at time step t
$\mathbf{h}_t \in \mathbb{R}^m$	Hidden state vector at time step t
$\mathbf{c}_t \in \mathbb{R}^m$	Cell state vector at time step t
$\mathbf{W} \in \mathbb{R}^{m \times n}$	Weight matrix for input transformations
$\mathbf{U} \in \mathbb{R}^{m \times m}$	Recurrent weight matrix
$\mathbf{b} \in \mathbb{R}^m$	Bias vector
$\sigma(\cdot)$	Element-wise sigmoid activation
$\tanh(\cdot)$	Element-wise hyperbolic tangent
$\odot$	Hadamard (element-wise) product

Table 1: Notation used throughout this document

2.2 Activation Functions

Definition 2.2 (Sigmoid Function)**.** The sigmoid function $\sigma : \mathbb{R} \to (0, 1)$ is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{1}$$

Definition 2.2 (Hyperbolic Tangent)**.** The hyperbolic tangent $\tanh : \mathbb{R} \to (-1, 1)$ is defined as:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1 \tag{2}$$

These functions satisfy the following derivative relationships, which are essential for back-propagation:

Proposition 2.3 (Activation Derivatives)**.** *The derivatives of the activation functions are:*

$$\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z)) \tag{3}$$

$$\frac{d\tanh}{dz} = 1 - \tanh^2(z) = \operatorname{sech}^2(z) \tag{4}$$

Proof. For the sigmoid function:

$$\begin{aligned}\frac{d\sigma}{dz} &= \frac{d}{dz}\left(\frac{1}{1+e^{-z}}\right) = \frac{e^{-z}}{(1+e^{-z})^2} \\ &= \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)(1-\sigma(z))\end{aligned}$$

The tanh derivative follows similarly or from the identity $\tanh(z) = 2\sigma(2z) - 1$. □

2.3 Hadamard Product

Definition 2.4 (Hadamard Product)**.** For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^m$, the Hadamard product $\mathbf{a} \odot \mathbf{b} \in \mathbb{R}^m$ is defined component-wise:

$$(\mathbf{a} \odot \mathbf{b})_j = a_j \cdot b_j, \quad j = 1, \ldots, m \tag{5}$$

Lemma 2.5 (Hadamard Product Properties)**.** *The Hadamard product satisfies:*

(i) Commutativity: $\mathbf{a} \odot \mathbf{b} = \mathbf{b} \odot \mathbf{a}$

(ii) Associativity: $(\mathbf{a} \odot \mathbf{b}) \odot \mathbf{c} = \mathbf{a} \odot (\mathbf{b} \odot \mathbf{c})$

(iii) Distributivity: $\mathbf{a} \odot (\mathbf{b} + \mathbf{c}) = \mathbf{a} \odot \mathbf{b} + \mathbf{a} \odot \mathbf{c}$

(iv) Matrix form: $\mathbf{a} \odot \mathbf{b} = \operatorname{diag}(\mathbf{a})\mathbf{b} = \operatorname{diag}(\mathbf{b})\mathbf{a}$

3 LSTM Cell Architecture

The LSTM cell consists of four main components: the *forget gate*, *input gate*, *cell state update*, and *output gate*. Each gate is parameterized by its own weight matrices and bias vectors.

3.1 The Forget Gate

The forget gate determines what information from the previous cell state should be discarded.

Definition 3.1 (Forget Gate)**.** The forget gate $\mathbf{f}_t \in (0,1)^m$ at time t is defined by:

$$\boxed{\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f)} \tag{6}$$

where $\mathbf{W}_f \in \mathbb{R}^{m \times n}$ is the input weight matrix, $\mathbf{U}_f \in \mathbb{R}^{m \times m}$ is the recurrent weight matrix, and $\mathbf{b}_f \in \mathbb{R}^m$ is the bias vector.

Remark 3.2 (Interpretation)**.** The forget gate acts as a multiplicative mask on the previous cell state. When $f_t^{(j)} \approx 0$, the j-th component of the cell state is "forgotten." When $f_t^{(j)} \approx 1$, the information is preserved. This selective forgetting mechanism allows the network to maintain relevant information over long sequences.

3.2 The Input Gate

The input gate controls what new information will be stored in the cell state. It consists of two parts: the input gate activation and the candidate cell state.

Definition 3.3 (Input Gate)**.** The input gate activation $\mathbf{i}_t \in (0,1)^m$ is defined by:

$$\boxed{\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i)} \tag{7}$$

where $\mathbf{W}_i \in \mathbb{R}^{m \times n}$, $\mathbf{U}_i \in \mathbb{R}^{m \times m}$, and $\mathbf{b}_i \in \mathbb{R}^m$ are the input gate parameters.

Definition 3.4 (Candidate Cell State)**.** The candidate cell state $\tilde{\mathbf{c}}_t \in (-1,1)^m$ is defined by:

$$\boxed{\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c)} \tag{8}$$

where $\mathbf{W}_c \in \mathbb{R}^{m \times n}$, $\mathbf{U}_c \in \mathbb{R}^{m \times m}$, and $\mathbf{b}_c \in \mathbb{R}^m$ are the candidate parameters.

The candidate $\tilde{\mathbf{c}}_t$ represents potential new information to be added to the cell state, while $\mathbf{i}_t$ determines how much of this information to actually incorporate.

3.3 Cell State Update

The cell state update is the core equation of the LSTM architecture.

Definition 3.5 (Cell State Update)**.** The cell state $\mathbf{c}_t \in \mathbb{R}^m$ is updated according to:

$$\boxed{\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t} \tag{9}$$

This equation combines:

- $\mathbf{f}_t \odot \mathbf{c}_{t-1}$: Filtered previous cell state (what to keep)
- $\mathbf{i}_t \odot \tilde{\mathbf{c}}_t$: Filtered candidate state (what to add)

Theorem 3.6 (Gradient Flow Property)**.** *The cell state update equation allows gradients to flow through time with minimal decay. Specifically, the Jacobian of* $\mathbf{c}_t$ *with respect to* $\mathbf{c}_{t-1}$ *is:*

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \operatorname{diag}(\mathbf{f}_t) + \textit{(terms depending on } \mathbf{f}_t, \mathbf{i}_t, \tilde{\mathbf{c}}_t\textit{)} \tag{10}$$

When the forget gate is close to $\mathbf{1}$ *(the vector of ones), the dominant term is* $\operatorname{diag}(\mathbf{f}_t) \approx \mathbf{I}$*, allowing gradients to flow almost unimpeded.*

Proof. Taking the partial derivative with respect to $\mathbf{c}_{t-1}$:

$$\begin{aligned}
\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} &= \frac{\partial}{\partial \mathbf{c}_{t-1}} \left[\mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t\right] \\
&= \operatorname{diag}(\mathbf{f}_t) + \mathbf{c}_{t-1} \odot \frac{\partial \mathbf{f}_t}{\partial \mathbf{c}_{t-1}} + \tilde{\mathbf{c}}_t \odot \frac{\partial \mathbf{i}_t}{\partial \mathbf{c}_{t-1}} + \mathbf{i}_t \odot \frac{\partial \tilde{\mathbf{c}}_t}{\partial \mathbf{c}_{t-1}}
\end{aligned}$$

In the standard LSTM (without peephole connections), the gates do not directly depend on $\mathbf{c}_{t-1}$, so the last three terms vanish, yielding $\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \operatorname{diag}(\mathbf{f}_t)$. □

3.4 The Output Gate

The output gate determines what parts of the cell state will be output as the hidden state.

Definition 3.7 (Output Gate)**.** The output gate $\mathbf{o}_t \in (0,1)^m$ is defined by:

$$\boxed{\mathbf{o}_t = \sigma(\mathbf{W}_o\mathbf{x}_t + \mathbf{U}_o\mathbf{h}_{t-1} + \mathbf{b}_o)} \tag{11}$$

where $\mathbf{W}_o \in \mathbb{R}^{m\times n}$, $\mathbf{U}_o \in \mathbb{R}^{m\times m}$, and $\mathbf{b}_o \in \mathbb{R}^m$ are the output gate parameters.

Definition 3.8 (Hidden State Output)**.** The hidden state $\mathbf{h}_t \in (-1,1)^m$ is computed as:

$$\boxed{\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)} \tag{12}$$

The tanh function squashes the cell state to the range $(-1,1)$, and the output gate filters which components are exposed as the hidden state.

4 Complete Forward Pass

4.1 Summary of LSTM Equations

The complete LSTM forward pass at time step t is given by the following system:

LSTM Forward Equations

$$\mathbf{f}_t = \sigma(\mathbf{W}_f\mathbf{x}_t + \mathbf{U}_f\mathbf{h}_{t-1} + \mathbf{b}_f) \quad \text{[Forget Gate]} \tag{13}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i\mathbf{x}_t + \mathbf{U}_i\mathbf{h}_{t-1} + \mathbf{b}_i) \quad \text{[Input Gate]} \tag{14}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c\mathbf{x}_t + \mathbf{U}_c\mathbf{h}_{t-1} + \mathbf{b}_c) \quad \text{[Candidate]} \tag{15}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad \text{[Cell Update]} \tag{16}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o\mathbf{x}_t + \mathbf{U}_o\mathbf{h}_{t-1} + \mathbf{b}_o) \quad \text{[Output Gate]} \tag{17}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{[Hidden State]} \tag{18}$$

4.2 Parameter Count

Proposition 4.1 (LSTM Parameter Count)**.** *For an LSTM with input dimension n and hidden dimension m, the total number of trainable parameters is:*

$$P = 4(nm + m^2 + m) = 4m(n + m + 1) \tag{19}$$

Proof. Each of the four gates (forget, input, candidate, output) has:

- Input weight matrix: $m \times n$ parameters
- Recurrent weight matrix: $m \times m$ parameters
- Bias vector: m parameters

Total: $4 \times (mn + m^2 + m) = 4m(n + m + 1)$. □

4.3 Initial Conditions

The LSTM requires initial values for the hidden state and cell state:

$$\mathbf{h}_0 = \mathbf{0} \in \mathbb{R}^m, \quad \mathbf{c}_0 = \mathbf{0} \in \mathbb{R}^m \tag{20}$$

These can also be learned parameters in some implementations.

5 Compact Matrix Formulation

For computational efficiency, the LSTM equations can be written in compact matrix form.

Definition 5.1 (Concatenated Parameters)**.** Define the concatenated weight matrices and bias vector:

$$\mathbf{W} = \begin{bmatrix} \mathbf{W}_f \\ \mathbf{W}_i \\ \mathbf{W}_c \\ \mathbf{W}_o \end{bmatrix} \in \mathbb{R}^{4m \times n} \tag{21}$$

$$\mathbf{U} = \begin{bmatrix} \mathbf{U}_f \\ \mathbf{U}_i \\ \mathbf{U}_c \\ \mathbf{U}_o \end{bmatrix} \in \mathbb{R}^{4m \times m} \tag{22}$$

$$\mathbf{b} = \begin{bmatrix} \mathbf{b}_f \\ \mathbf{b}_i \\ \mathbf{b}_c \\ \mathbf{b}_o \end{bmatrix} \in \mathbb{R}^{4m} \tag{23}$$

Proposition 5.2 (Efficient Computation)**.** *The pre-activations for all gates can be computed in a single matrix operation:*

$$\boxed{\mathbf{z}_t = \mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b} \in \mathbb{R}^{4m}} \tag{24}$$

The vector $\mathbf{z}_t$ is then split into four segments of size m:

$$\mathbf{f}_t = \sigma(\mathbf{z}_t^{[1:m]}) \tag{25}$$

$$\mathbf{i}_t = \sigma(\mathbf{z}_t^{[m+1:2m]}) \tag{26}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{z}_t^{[2m+1:3m]}) \tag{27}$$

$$\mathbf{o}_t = \sigma(\mathbf{z}_t^{[3m+1:4m]}) \tag{28}$$

This formulation reduces the number of matrix multiplications from 8 to 2, significantly improving computational efficiency.

6 Backpropagation Through Time

Given a loss function $\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t(\mathbf{h}_t, \mathbf{y}_t)$ where $\mathbf{y}_t$ are target values, we derive the gradients using the chain rule.

6.1 Gradient with Respect to Hidden State

Define the error signal at the hidden state:

$$\boldsymbol{\delta}_t^h = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \tag{29}$$

This receives contributions from:

(i) The loss at time t: $\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t}$

(ii) Backpropagation from time $t+1$: through gates at $t+1$ that depend on $\mathbf{h}_t$

6.2 Gradient with Respect to Cell State

Theorem 6.1 (Cell State Gradient Recurrence)**.** *Define* $\boldsymbol{\delta}_t^c = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t}$*. The recurrence relation is:*

$$\boxed{\boldsymbol{\delta}_t^c = \boldsymbol{\delta}_t^h \odot \mathbf{o}_t \odot (1 - \tanh^2(\mathbf{c}_t)) + \boldsymbol{\delta}_{t+1}^c \odot \mathbf{f}_{t+1}} \tag{30}$$

Proof. By the chain rule:

$$\boldsymbol{\delta}_t^c = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t}\frac{\partial \mathbf{h}_t}{\partial \mathbf{c}_t} + \frac{\partial \mathcal{L}}{\partial \mathbf{c}_{t+1}}\frac{\partial \mathbf{c}_{t+1}}{\partial \mathbf{c}_t}$$

Since $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{c}_t} = \operatorname{diag}(\mathbf{o}_t)\operatorname{diag}(1 - \tanh^2(\mathbf{c}_t))$$

And since $\mathbf{c}_{t+1} = \mathbf{f}_{t+1} \odot \mathbf{c}_t + \mathbf{i}_{t+1} \odot \tilde{\mathbf{c}}_{t+1}$:

$$\frac{\partial \mathbf{c}_{t+1}}{\partial \mathbf{c}_t} = \operatorname{diag}(\mathbf{f}_{t+1})$$

Combining these yields the recurrence. □

Corollary 6.2 (Vanishing Gradient Mitigation)**.** *When* $\mathbf{f}_t \approx \mathbf{1}$ *for all* t*, the gradient satisfies:*

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_1} \approx \frac{\partial \mathcal{L}}{\partial \mathbf{c}_T}\prod_{t=2}^{T}\operatorname{diag}(\mathbf{f}_t) \approx \frac{\partial \mathcal{L}}{\partial \mathbf{c}_T} \tag{31}$$

This prevents the exponential decay characteristic of vanilla RNNs.

6.3 Gate Gradients

The gradients for each gate pre-activation are:

Gate Gradient Formulas

$$\boldsymbol{\delta}_t^f = \boldsymbol{\delta}_t^c \odot \mathbf{c}_{t-1} \odot \mathbf{f}_t \odot (1 - \mathbf{f}_t) \quad \text{[Forget Gate]} \tag{32}$$

$$\boldsymbol{\delta}_t^i = \boldsymbol{\delta}_t^c \odot \tilde{\mathbf{c}}_t \odot \mathbf{i}_t \odot (1 - \mathbf{i}_t) \quad \text{[Input Gate]} \tag{33}$$

$$\boldsymbol{\delta}_t^{\tilde{c}} = \boldsymbol{\delta}_t^c \odot \mathbf{i}_t \odot (1 - \tilde{\mathbf{c}}_t^2) \quad \text{[Candidate]} \tag{34}$$

$$\boldsymbol{\delta}_t^o = \boldsymbol{\delta}_t^h \odot \tanh(\mathbf{c}_t) \odot \mathbf{o}_t \odot (1 - \mathbf{o}_t) \quad \text{[Output Gate]} \tag{35}$$

6.4 Weight Gradients

The gradients with respect to weights are accumulated over all time steps:

Theorem 6.3 (Weight Gradient Formulas)**.** *The gradients with respect to the forget gate parameters are:*

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_f} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^f \mathbf{x}_t^\top \tag{36}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{U}_f} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^f \mathbf{h}_{t-1}^\top \tag{37}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_f} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^f \tag{38}$$

Analogous expressions hold for the input gate, candidate, and output gate parameters.

7 LSTM Variants

7.1 Peephole Connections

Peephole connections, introduced by Gers and Schmidhuber [2], allow gates to access the cell state directly:

LSTM with Peephole Connections

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{V}_f \odot \mathbf{c}_{t-1} + \mathbf{b}_f) \tag{39}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{V}_i \odot \mathbf{c}_{t-1} + \mathbf{b}_i) \tag{40}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{V}_o \odot \mathbf{c}_t + \mathbf{b}_o) \tag{41}$$

where $\mathbf{V}_f, \mathbf{V}_i, \mathbf{V}_o \in \mathbb{R}^m$ are diagonal peephole weight vectors (typically implemented as element-wise multiplication).

7.2 Gated Recurrent Unit (GRU)

The GRU, introduced by Cho et al. [3], is a simplified variant that combines the forget and input gates:

GRU Equations

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r) \quad \text{[Reset Gate]} \tag{42}$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z) \quad \text{[Update Gate]} \tag{43}$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h) \quad \text{[Candidate]} \tag{44}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad \text{[Hidden State]} \tag{45}$$

Remark 7.1. The GRU has fewer parameters than the LSTM (3 gates vs 4) and no separate cell state. Despite this simplification, GRUs often achieve comparable performance on many tasks.

7.3 Coupled Input-Forget Gate

A variant that enforces $\mathbf{i}_t = 1 - \mathbf{f}_t$:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + (1 - \mathbf{f}_t) \odot \tilde{\mathbf{c}}_t \tag{46}$$

This reduces the parameter count while maintaining the core LSTM functionality.

8 Bidirectional LSTM

Definition 8.1 (Bidirectional LSTM)**.** A bidirectional LSTM processes sequences in both directions:

$$\overrightarrow{\mathbf{h}}_t = \mathrm{LSTM}_{\mathrm{forward}}(\mathbf{x}_t, \overrightarrow{\mathbf{h}}_{t-1}, \overrightarrow{\mathbf{c}}_{t-1}) \tag{47}$$

$$\overleftarrow{\mathbf{h}}_t = \mathrm{LSTM}_{\mathrm{backward}}(\mathbf{x}_t, \overleftarrow{\mathbf{h}}_{t+1}, \overleftarrow{\mathbf{c}}_{t+1}) \tag{48}$$

The outputs are concatenated:

$$\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \in \mathbb{R}^{2m} \tag{49}$$

The bidirectional architecture allows the network to capture both past and future context at each time step.

9 Multi-Layer (Stacked) LSTM

Definition 9.1 (Stacked LSTM)**.** For an L-layer LSTM, let $\mathbf{h}_t^{(\ell)}$ denote the hidden state at time t and layer ℓ. The input to layer ℓ is the hidden state from layer $\ell - 1$:

$$\mathbf{h}_t^{(1)} = \mathrm{LSTM}^{(1)}(\mathbf{x}_t, \mathbf{h}_{t-1}^{(1)}, \mathbf{c}_{t-1}^{(1)}) \tag{50}$$

$$\mathbf{h}_t^{(\ell)} = \mathrm{LSTM}^{(\ell)}(\mathbf{h}_t^{(\ell-1)}, \mathbf{h}_{t-1}^{(\ell)}, \mathbf{c}_{t-1}^{(\ell)}), \quad \ell = 2, \ldots, L \tag{51}$$

Each layer has its own set of parameters, and the total parameter count scales linearly with the number of layers.

10 Conclusion

The LSTM architecture provides a mathematically elegant solution to the vanishing gradient problem through its gating mechanisms and additive cell state update. The key mathematical insight is that the cell state update equation

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \tag{52}$$

creates a linear path for gradients to flow through time when $\mathbf{f}_t \approx \mathbf{1}$, enabling the learning of long-range dependencies.

The mathematical framework presented in this document provides the foundation for understanding, implementing, and extending LSTM architectures for various sequence modeling tasks including natural language processing, speech recognition, time series forecasting, and beyond.

References

[1] Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory.* Neural Computation, 9(8), 1735–1780.

[2] Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). *Learning to forget: Continual prediction with LSTM.* Neural Computation, 12(10), 2451–2471.

[3] Cho, K., van Merrienboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning phrase representations using RNN encoder-decoder for statistical machine translation.* In Proceedings of EMNLP.

[4] Greff, K., Srivastava, R. K., Koutník, J., Steunebrink, B. R., & Schmidhuber, J. (2017). *LSTM: A search space odyssey.* IEEE Transactions on Neural Networks and Learning Systems, 28(10), 2222–2232.

[5] Graves, A. (2013). *Generating sequences with recurrent neural networks.* arXiv preprint arXiv:1308.0850.